

N-gram from scratch (2)

todo: 멋진 것 만들기

뉴비..를 위한 인공지능이 재밌어지는 시간! - 만들어야 하는 것

```
import random

def beautiful_f(x):
    return random.randint(0, total_tokens)

def generate(x, function, max_length=100):
    for i in range(len(x), max_length):
        x += stringify([function(x[i-n:i])])
    return x

chunk = chunks[0]
x = chunk[:n]
y = chunk[n]
print(generate(x, beautiful_f))
```

✓ 0.0s

날개
봉개왜집궁하새월형냐며덩못운젊코괘뜰넛럽막든면센너퍼뚜곽릿냐막싫페흘팩싸특조
람많멩찌복써써커제림뻘젼랴문엣넬쏘앞닿띄칸뜸런곰들뜨왔튼룩떡논알용털지잉세센참칸큼줄릴듯색엣느겸1만_덜래라걱엣썩뚜

beautiful_f를 우리가 직접 짜는 것..
(*매우 천재만 가능)

수학의 힘을 빌리자!

뉴비..를 위한 인공지능이 재밌어지는 시간!

1. **output을 next token 확률분포로**
2. **embedding을 사용하자!**

<http://bit.ly/ngramyoonhero2>

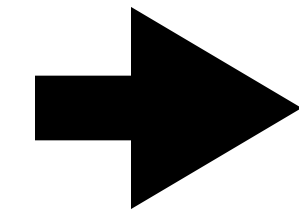
뉴비..를 위한 인공지능이 재밌어지는 시간!

저번 시간에 주구장창 한 것

1. 모델을 설계함

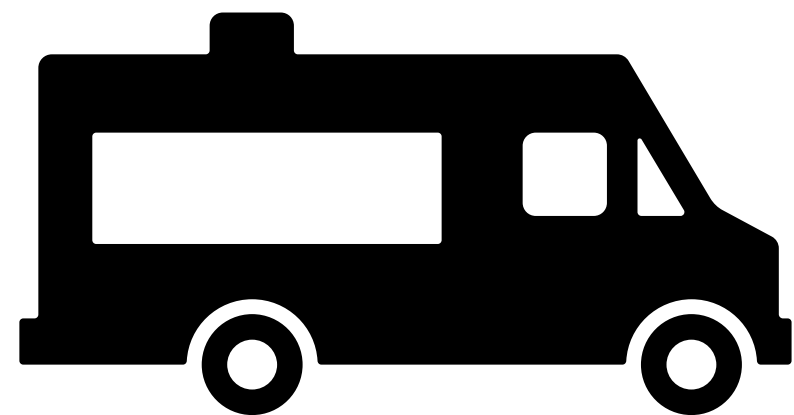
2. dL/dw 오차 역전파를 손으로 계산한다.

2. 이 과정을 계속 반복하는 loop를 만든다.



pytorch/tensorflow 딸깍

(제가 미분해드릴게요.)



뉴비...를 위한 인공지능이 재밌어지는 시간!

(제가 미분해드릴게요.)

```
w = torch.tensor([1, 2, 3], requires_grad=True, dtype=torch.float)
x = torch.tensor([2, 1, 3], requires_grad=True, dtype=torch.float)
y = w @ x
✓ 0.0s

w.grad
✓ 0.0s

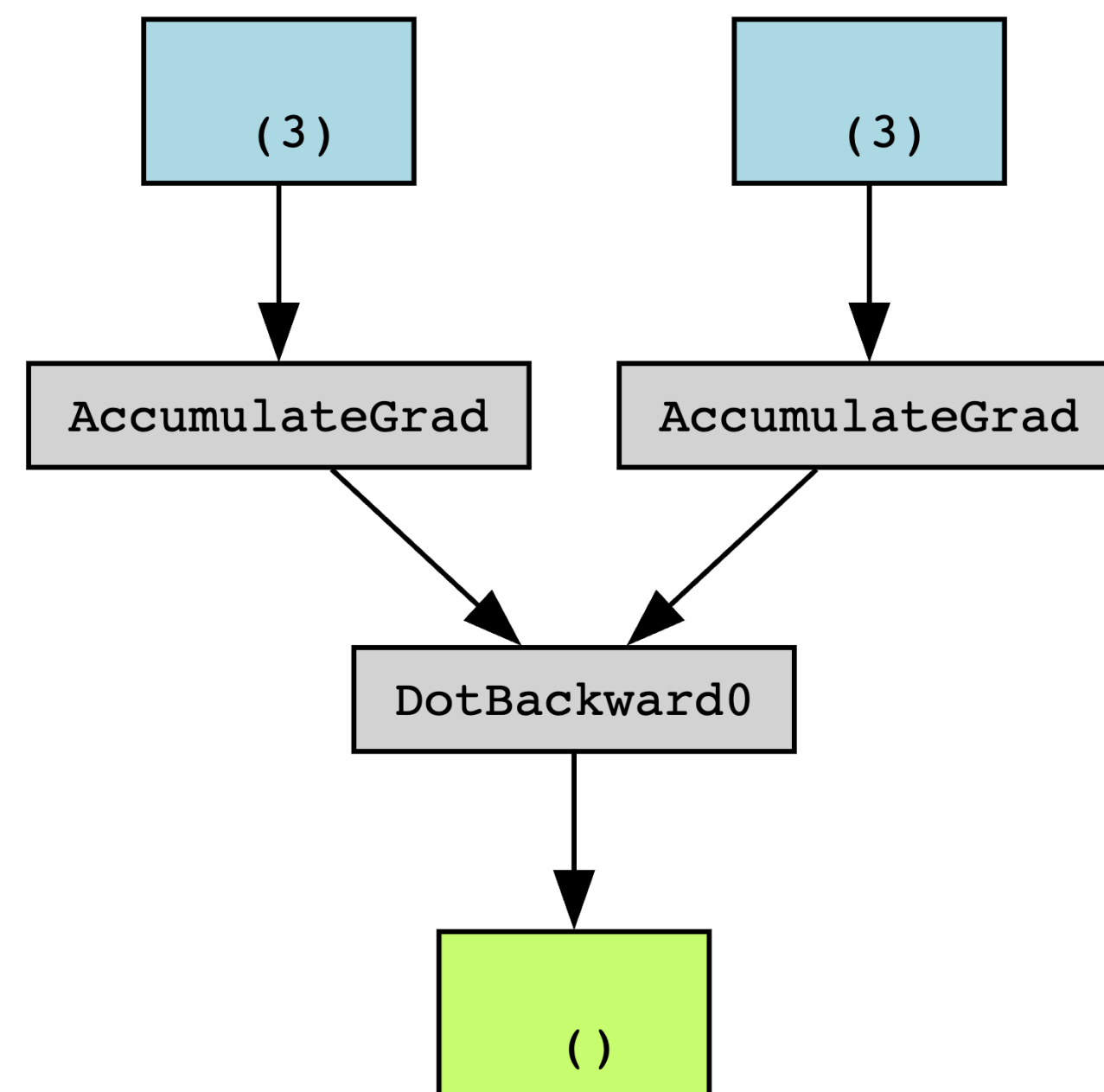
y.backward()
w.grad
✓ 0.0s

tensor([2., 1., 3.])
```

**backward를 호출할 시에
오차역전파를 자동으로
계산해줌!**

뉴비..를 위한 인공지능이 재밌어지는 시간!

마법이 아니다..(아주 간략한 설명)



topological order을 따라가게 됨!

```
class MyAdd(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, y):
        ctx.save_for_backward(x, y)
        return x + y

    @staticmethod
    def backward(ctx, grad_out):
        x, y = ctx.saved_tensors
        return grad_out * torch.ones_like(x), grad_out * torch.ones_like(y)

class my_tensor():
    def __init__(self, t):
        self.t = t
    def __add__(self, x):
        return MyAdd.apply(self.t, x.t)
```

이런식으로 연산을 할 때마다 graph를 만듦!

뉴비..를 위한 인공지능이 재밌어지는 시간!

지난 시간에 numpy 손코딩을 pytorch로 바꾸어보자...

```
function Luamlp:New(ni, nh1, nh2, nout)

-----
-- Function Back-Propagation
-- Parameters: y -> table (of expected outputs n training)
-----

function mlp:Backpropagation(y)

    local y = y or nil

    local out_delta = Narray(1, self.nout, 0)
    local nerror = 0.0
    -- value of return --
    local lms_error = 0.0

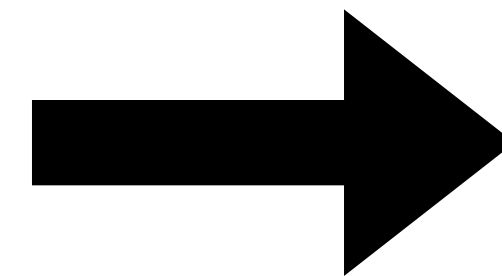
    -- error lms algorithm (return of function)
    for i = 1, #y do
        lms_error = lms_error + (0.5 * ((y[i] - self.yout[i]) ^ 2))
    end

    -- outputs delts calcule
    for i = 1, self.nout do

        nerror = y[i] - self.yout[i]
        out_delta[i] = self.dfx(self.sum_out[i]) * nerror

    end

end
```



loss.backward()

real 테토=lua backprop pytorch 시절

뉴비..를 위한 인공지능이 재밌어지는 시간!

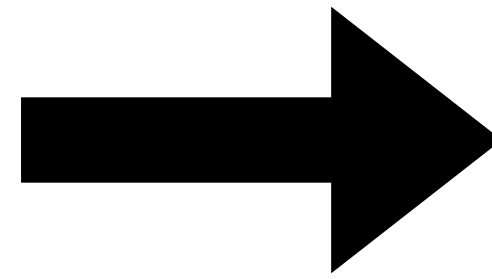
지난 시간에 numpy 손코딩을 pytorch로 바꾸어보자...

```
lr = 1e-3
```

```
for p in parameters:
```

```
    p.data -= lr * p.grad
```

```
    p.grad = None
```



```
optimizer = optim.SGD(parameters, lr=1e-3)
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
optimizer.step()
```

뉴비..를 위한 인공지능이 재밌어지는 시간!

지난 시간에 numpy 손코딩을 pytorch로 바꾸어보자...

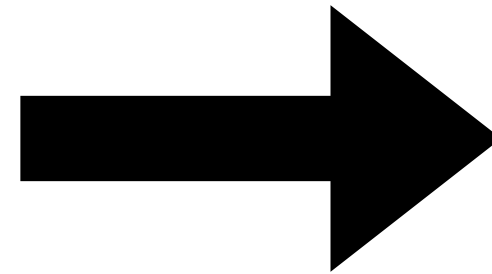
```
x = Emb[inds].reshape(small_batch, -1)
```

```
y = x @ W + B
```

```
exped = torch.exp(y-y.max())
```

```
softmaxed = exped / torch.sum(exped, axis=-1).view(-1, 1)
```

```
loss = (-torch.log2(softmaxed[range(small_batch), gts])).mean()
```



```
my_first_model = nn.Sequential(  
    nn.Embedding(total_tokens, emb_size),  
    nn.Flatten(),  
    nn.Linear(emb_size*n, total_tokens)  
)  
y = my_first_model(inds)  
loss = F.cross_entropy(y, gts)
```

뉴비..를 위한 인공지능이 재밌어지는 시간!

지난 시간에 numpy 손코딩을 pytorch로 바꾸어보자...

nn.Sequential -> 순서대로 실행되는 간단한 모델이라면..

nn.ModuleList -> Module을 List로..

nn.Module -> 많은 경우 내가 원하는 연산을 수행하는 모듈을 만드는 편

```
class SmallNetwork(nn.Module):
    def __init__(self): # 64 -> 32 -> 16 -> 8
        super().__init__()
        self.projection0 = nn.Linear(3*64*64, 512)
        self.projection1 = nn.Linear(512, 10)

        self.act = nn.ReLU()

    def forward(self, x, y=None):
        B = x.size(0)
        x = x.view(B, -1)
        x = self.projection0(x)
        x = self.act(x)
        x = self.projection1(x)
        loss = None
        if y is not None:
            loss = F.cross_entropy(x, y, reduction="mean")
        return x, loss
```


뉴비..를 위한 인공지능이 재밌어지는 시간!

Universal approximation Theorem

compact K 위에서 연속 함수 $f(x)$

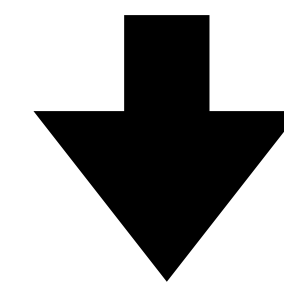
there exists $k \in \mathbb{N}, A \in \mathbb{R}^{k \times n}, b \in \mathbb{R}^k, C \in \mathbb{R}^{m \times k}$

s.t. $\sup_{x \in K} |f(x) - g(x)| < \epsilon$

where, $g(x) = C(\sigma(Ax + b))$

흥미로운 지점

은닉층이 하나여도 $\neg$ 첫
효율적인 것은 아님

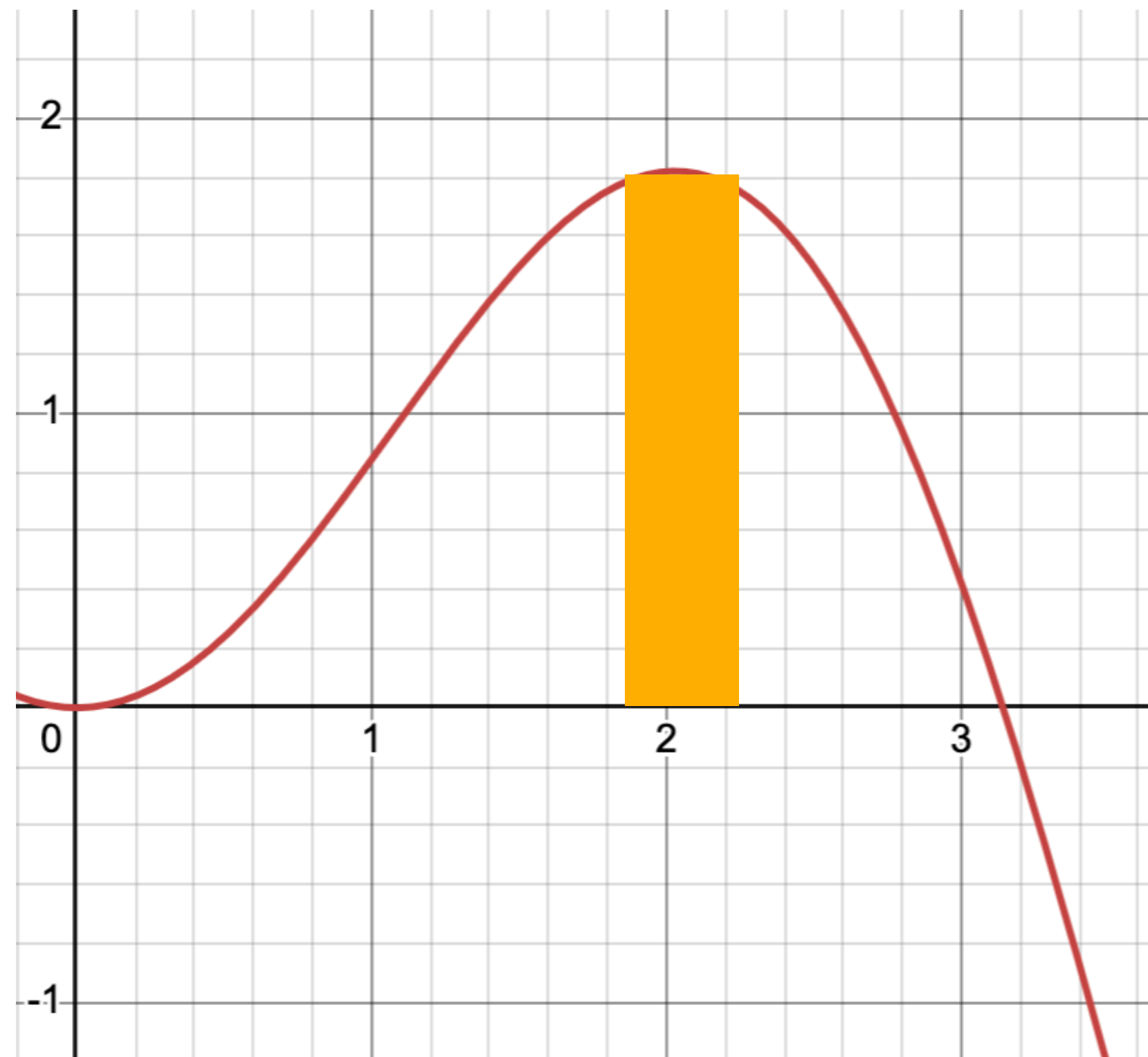


Engineering이 필요한 이유

https://en.wikipedia.org/wiki/Universal_approximation_theorem

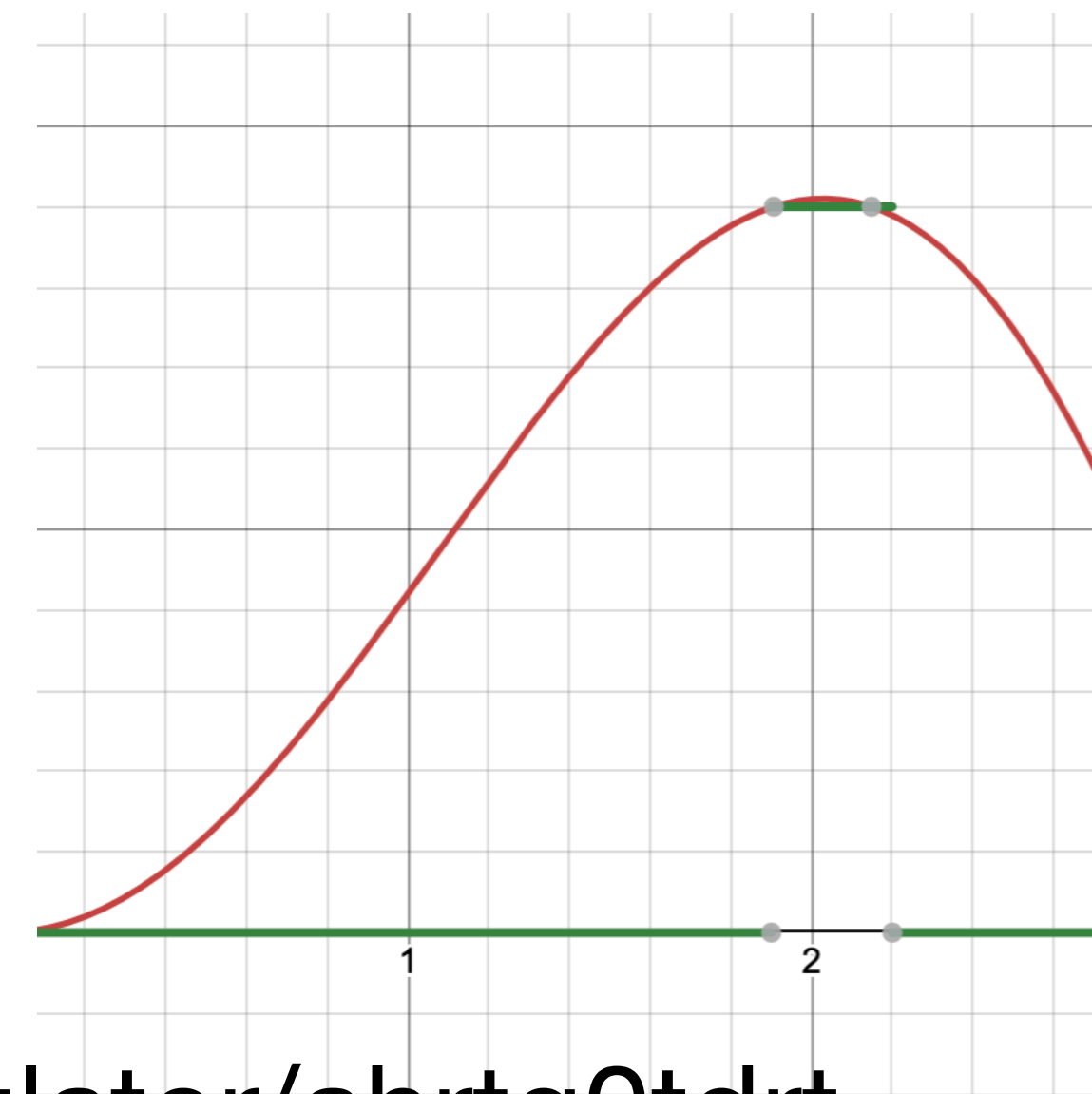
뉴비..를 위한 인공지능이 재밌어지는 시간!

마치 푸리에 급수처럼!



$\sigma = \min(0, \text{sign}(x))$: step function

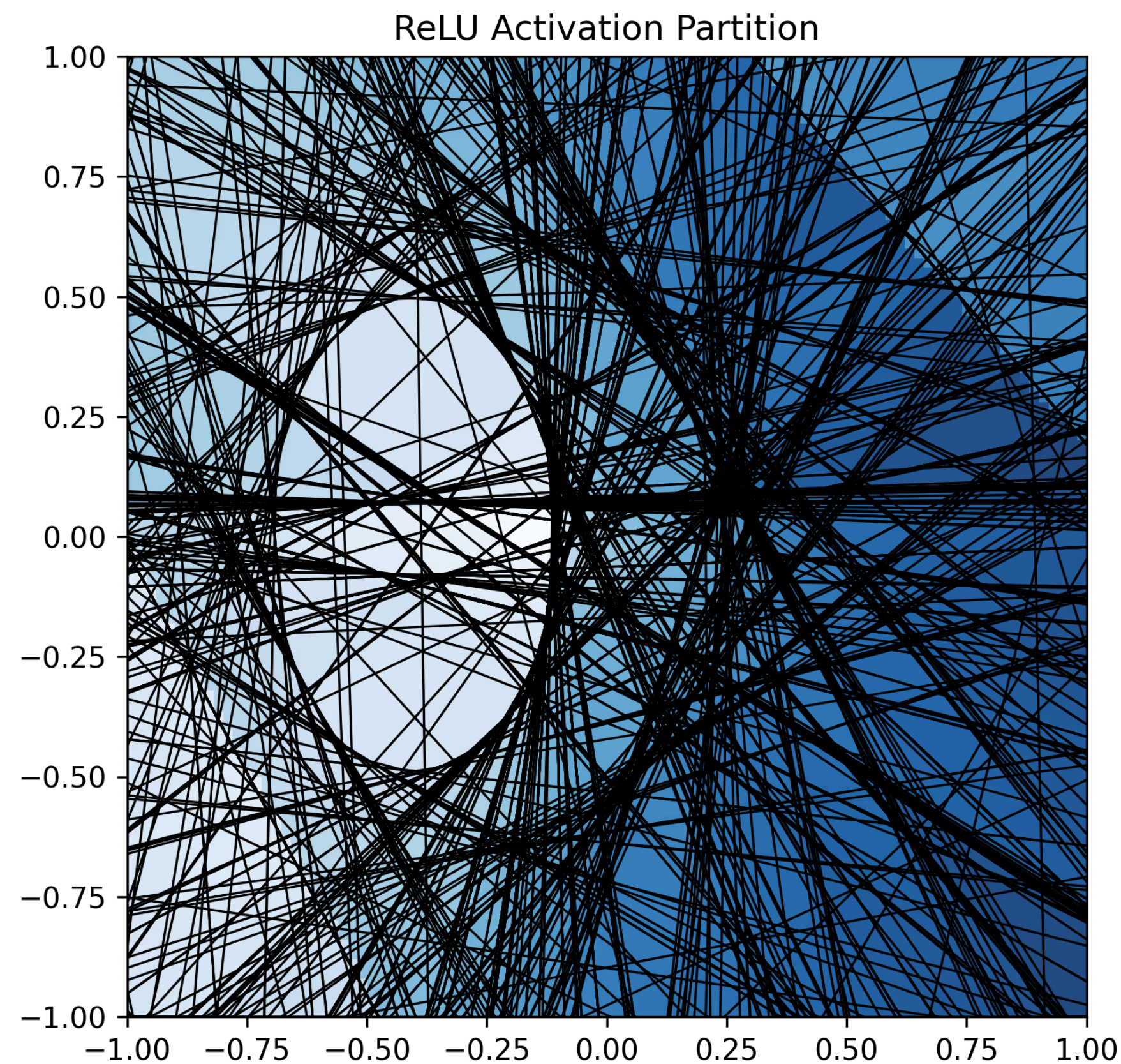
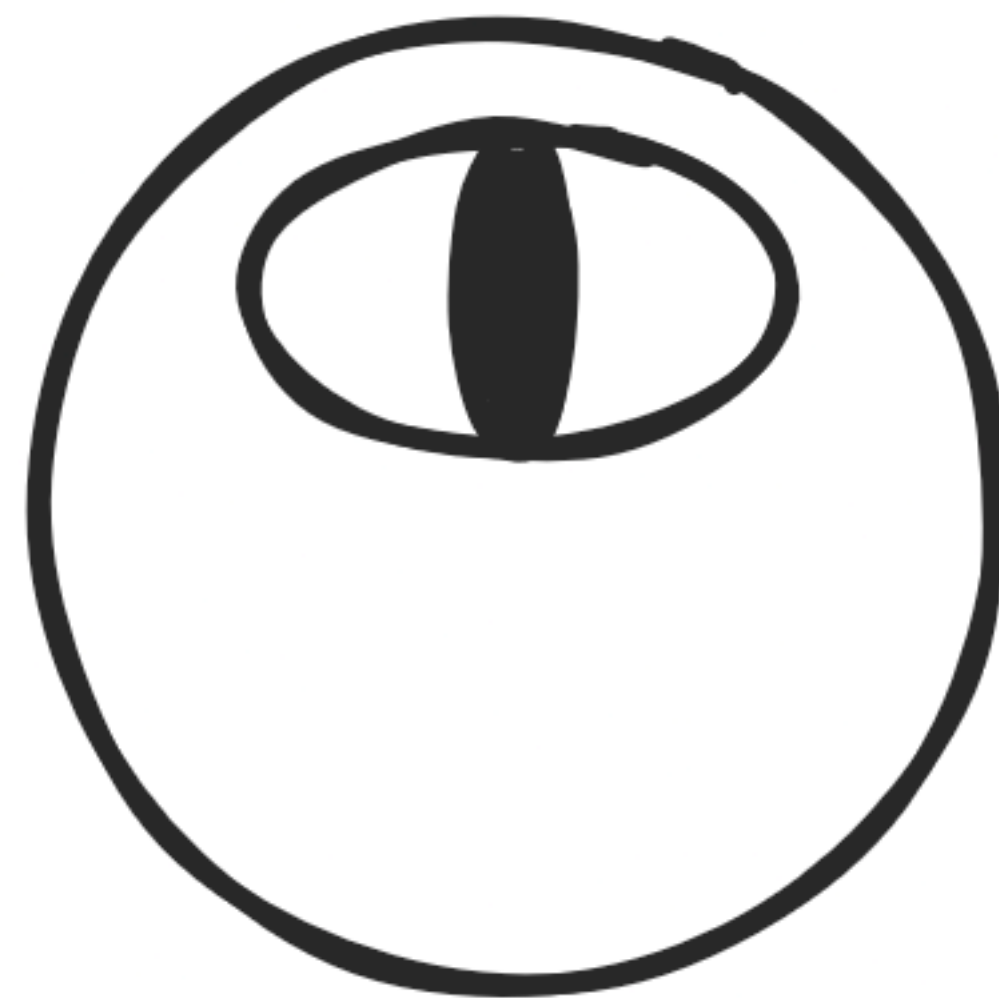
$$1.8(\sigma(x - 1.9) - \sigma(x - 2.2))$$



<https://www.desmos.com/calculator/ahrtq0tdrt>

https://en.wikipedia.org/wiki/Universal_approximation_theorem

뉴비..를 위한 인공지능이 재밌어지는 시간!



선형 함수가 만들지 못하는
데이터의 모양을 “흉내”낼 수 있다.
~함수의 표현력

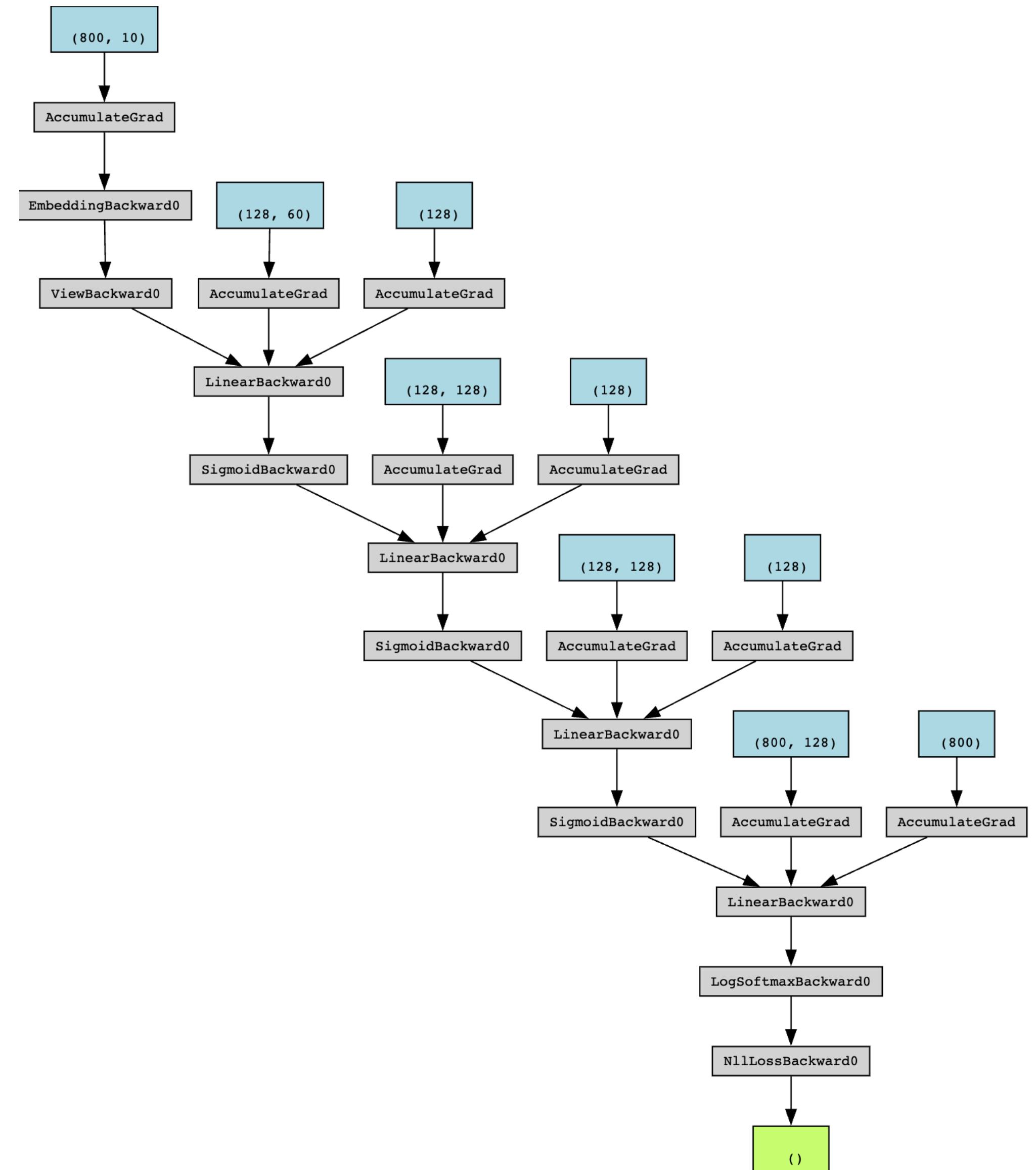
뉴비...를 위한 인공지능이 재밌어지는 시간!

2025 윤승현

N-gram from scratch (2)

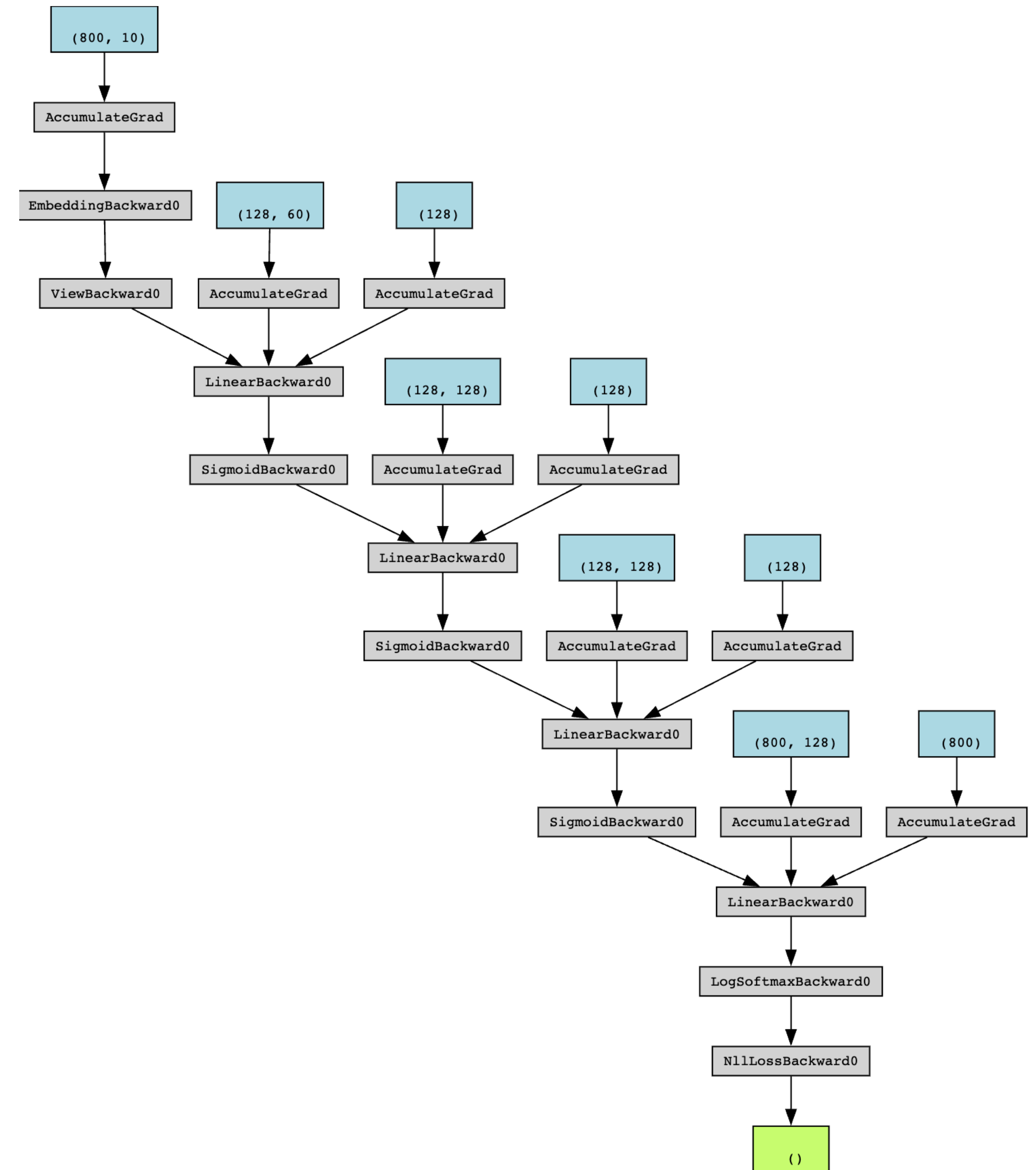
뉴비...를 위한 인공지능이 재밌어지는 시간!

```
my_first_model = nn.Sequential(  
    nn.Embedding(total_tokens, 10),  
    nn.Flatten(),  
    nn.Linear(10*n, 128),  
    nn.Sigmoid(),  
    nn.Linear(128, 128),  
    nn.Sigmoid(),  
    nn.Linear(128, 128),  
    nn.Sigmoid(),  
    nn.Linear(128, total_tokens)  
)
```



뉴비..를 위한 인공지능이 재밌어지는 시간!

'박제가_되어버린_것이_앓아_잘난_물아_아간_보
았다._그_오원_돈을_꺼내_아내_손에_주어지는_
것이_있었다._나는_이불을_뒤집어썼다._한_번도_
_구경한_일이_없으나_언제든지_끼니_때면_내_방
으로_가서_쪽_빠진_옷을_활활_벗어_버리고도_싶
었다.\n나는_내_잘못된_생각을_거기저_푹마한_
것이다._그러나_그것은_이불_속에_있었다._나는_
_이불을_뒤집어썼다._한_번도_구경한_일이_없으
나_언제든지_끼니_때면_내_방으로_가서_쪽_빠진_
_옷을_활활_벗어_버리고도_싶었다.\n나는_내_잘
못된_생각을_거기저_푹마한_것이다._그러나_그
것은_이불_속에_있었다._나는_'



뉴비..를 위한 인공지능이 재밌어지는 시간! - context 창 길이를 어떻게 무한대로?

다음 시간에 계속....